**GreenCart — Dockerized 3-Tier Application**

A full-stack grocery e-commerce application containerized and deployed as a 3-tier Docker Compose application on an AWS EC2 instance.

**Project Overview**

GreenCart is a grocery shopping application consisting of:

- **Frontend:** React + Vite
- **Backend:** Node.js + Express
- **Database:** MongoDB
- **External Services:** Cloudinary and Stripe integrations
- **Deployment:** Docker Compose on AWS EC2

The original application was not Dockerized before this project. Docker support, container configuration, Compose orchestration, networking, persistent storage, and EC2 deployment were added as part of this project.

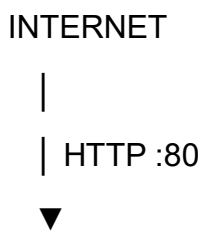
**Original Application**

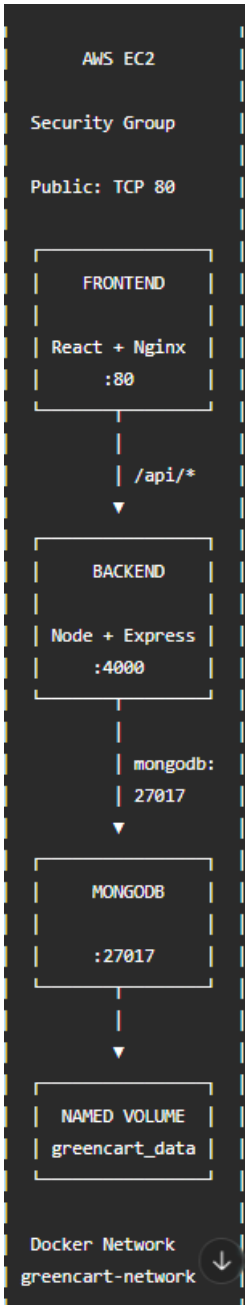
**Original GitHub Repository:**

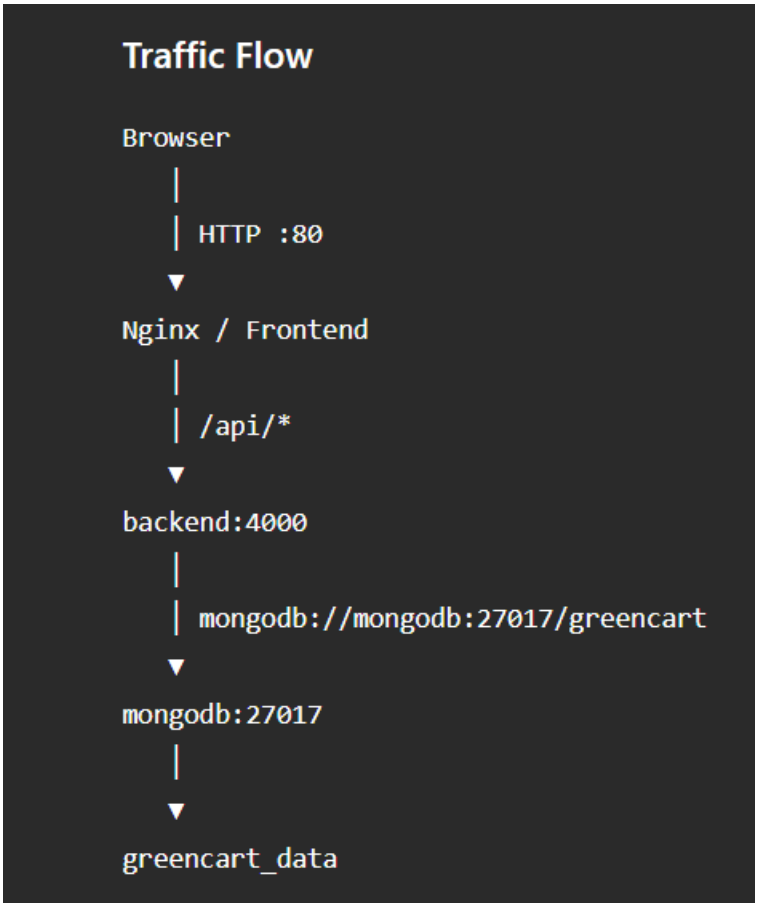
https://github.com/therani-2006/GreenCart

The original repository was used as the application source before Dockerization.

**Architecture**




AWS EC2
Security Group
Public: TCP 80
FRONTEND
React + Nginx
:80
/api/*
BACKEND
Node + Express
:4000
mongodb:
27017
MONGODB
:27017
NAMED VOLUME
greencart_data
Docker Network
greencart-network




**Public vs Internal Ports**

| Service | Container Port | Public/EC2 Port | Access |
|---|---|---|---|
| Frontend / Nginx | 80 | 80 | Public |
| Backend / Express | 4000 | Not exposed | Internal |
| MongoDB | 27017 | Not exposed | Internal |

Only port **80** is exposed to the Internet.

The backend and MongoDB communicate exclusively through the Docker network.

**Docker Configuration**

**Backend**

The backend uses a multi-stage Docker build based on Node.js Alpine.

FROM node:22-alpine AS builder

```
WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

FROM node:22-alpine AS production

WORKDIR /app

ENV NODE_ENV=production
ENV PORT=4000

COPY --from=builder /app/node_modules ./node_modules
COPY package*.json ./
COPY . .

EXPOSE 4000

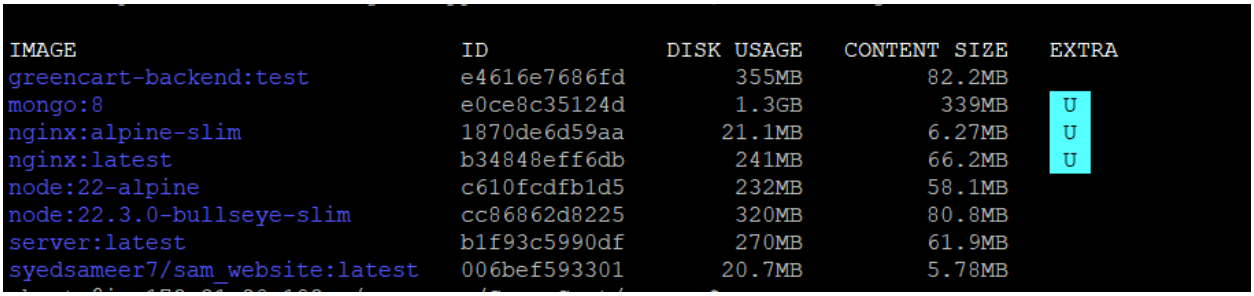
USER node

CMD ["npm", "start"]
```

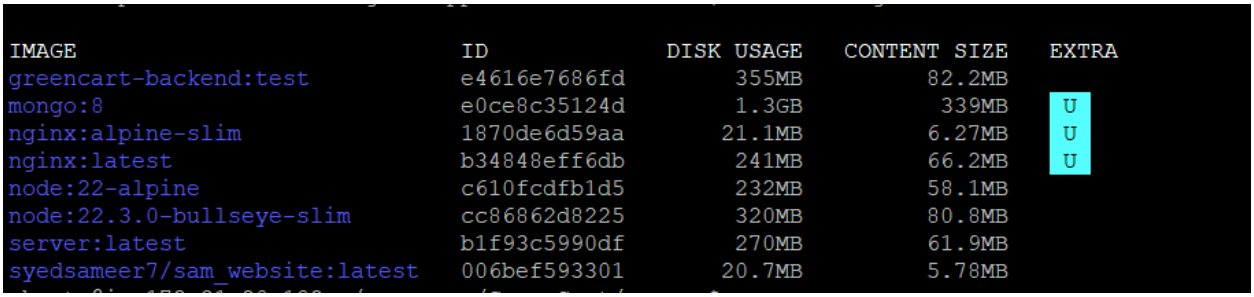
**Backend Optimization**

The backend Dockerfile uses:

- Multi-stage builds
- node:22-alpine
- Production-only npm dependencies
- .dockerignore
- Non-root node user
- No development dependencies in the production image



| IMAGE | ID | DISK USAGE | CONTENT SIZE | EXTRA |
|---|---|---|---|---|
| greencart-backend:test | e4616e7686fd | 355MB | 82.2MB | |
| mongo:8 | e0ce8c35124d | 1.3GB | 339MB | U |
| nginx:alpine-slim | 1870de6d59aa | 21.1MB | 6.27MB | U |
| nginx:latest | b34848eff6db | 241MB | 66.2MB | U |
| node:22-alpine | c610fcdfb1d5 | 232MB | 58.1MB | |
| node:22.3.0-bullseye-slim | cc86862d8225 | 320MB | 80.8MB | |
| server:latest | b1f93c5990df | 270MB | 61.9MB | |
| syedsameer7/sam_website:latest | 006bef593301 | 20.7MB | 5.78MB | |



| IMAGE | ID | DISK USAGE | CONTENT SIZE | EXTRA |
|---|---|---|---|---|
| greencart-backend:test | e4616e7686fd | 355MB | 82.2MB | |
| mongo:8 | e0ce8c35124d | 1.3GB | 339MB | U |
| nginx:alpine-slim | 1870de6d59aa | 21.1MB | 6.27MB | U |
| nginx:latest | b34848eff6db | 241MB | 66.2MB | U |
| node:22-alpine | c610fcdfb1d5 | 232MB | 58.1MB | |
| node:22.3.0-bullseye-slim | cc86862d8225 | 320MB | 80.8MB | |
| server:latest | b1f93c5990df | 270MB | 61.9MB | |
| syedsameer7/sam_website:latest | 006bef593301 | 20.7MB | 5.78MB | |

**Frontend**

The frontend is built using Vite and served using Nginx.

The Dockerfile uses a build stage to compile the React application and a lightweight Nginx runtime image to serve the generated static files.

This separates build dependencies from the production runtime.

**Frontend Optimization**

The frontend uses:

- Multi-stage Docker build
- Node Alpine for the build stage
- Nginx for the production runtime

- .dockerignore
- Production Vite build
- Static asset serving through Nginx

**Docker Compose**

All three services are orchestrated using one docker-compose.yml file.

Services:

mongodb

backend

frontend

The services share a custom Docker bridge network:

greencart-network

**Service Communication**

The backend does not connect to MongoDB using localhost.

Instead, Docker's internal DNS resolves:

mongodb

to the MongoDB container.

The backend connection string is:

mongodb://mongodb:27017/greencart

Similarly, the frontend/Nginx container routes API requests internally to:

backend:4000

This avoids hardcoded container IP addresses.

**Persistent Database Storage**

MongoDB uses a named Docker volume:

greencart_data

The volume is mounted at:

/data/db

Configuration:

volumes:

- greencart_data:/data/db

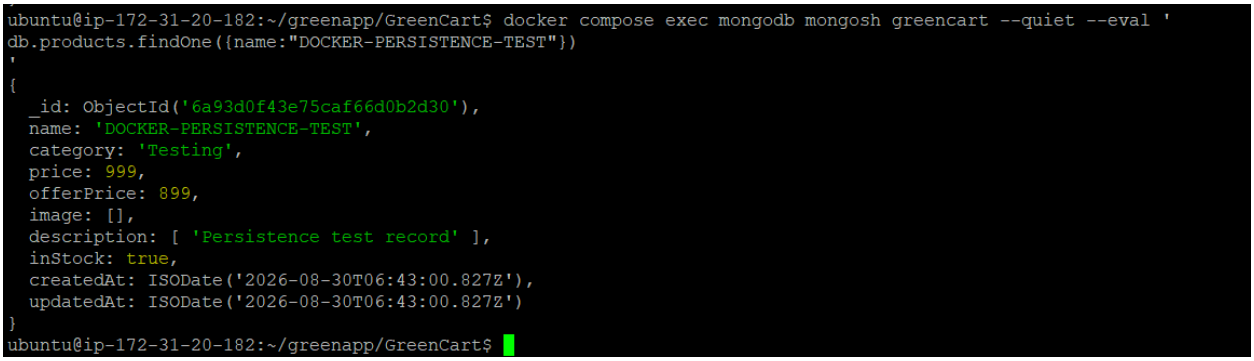
**Persistence Verification**

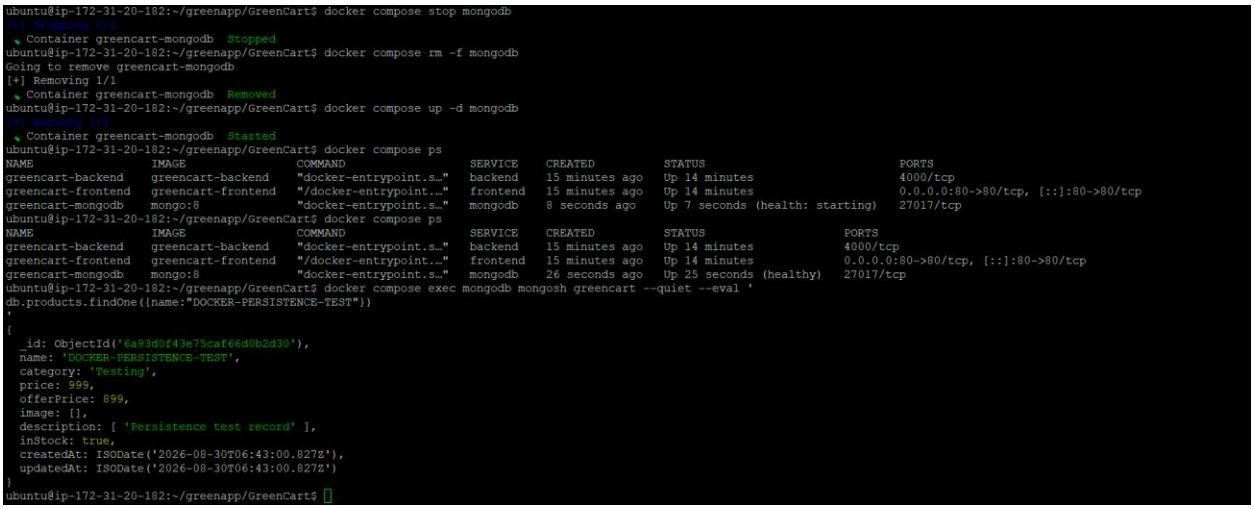
Persistence was tested by:

1. Creating a test record in MongoDB.
2. Verifying that the record existed.
3. Stopping the MongoDB container.
4. Removing the MongoDB container.
5. Recreating the MongoDB container.
6. Querying the database again.
7. Confirming that the original record was still present.

The data survived MongoDB container recreation because the database files are stored in the named Docker volume rather than inside the container filesystem.

Persistence test screenshots are included in the deployment evidence document.






**Environment Variables and Secrets**

Secrets are not committed to GitHub.

The repository .gitignore excludes:

.env

.env.local

.env.development.local

.env.test.local

.env.production.local

Docker Compose receives sensitive configuration through environment variables.

Examples include:

JWT_SECRET

SELLER_EMAIL

SELLER_PASSWORD

CLOUDINARY_CLOUD_NAME

CLOUDINARY_API_KEY

CLOUDINARY_API_SECRET

STRIPE_SECRET_KEY

STRIPE_WEBHOOK_SECRET

The MongoDB connection string used by the Docker deployment is configured internally by Compose:

mongodb://mongodb:27017/greencart

**Image Sizes**

Final application image sizes measured from Docker:

| Image | Size |
|---|---|
| greencart-frontend:latest | **107 MB** |
| greencart-backend:latest | **270 MB** |

MongoDB uses the official:

mongo:8

image.

The MongoDB base image is approximately **1.3 GB** and is not a custom application image.

**Running the Application**

**Prerequisites**

- Docker
- Docker Compose
- Git
- AWS EC2 instance for cloud deployment

**Clone the Repository**

git clone https://github.com/sameersyed7/GreenCart.git

cd GreenCart

**Configure Environment Variables**

Create a .env file in the project root.

Example:

JWT_SECRET=<your-secret>

SELLER_EMAIL=<your-seller-email>

SELLER_PASSWORD=<your-seller-password>

CLOUDINARY_CLOUD_NAME=<your-cloud-name>

CLOUDINARY_API_KEY=<your-api-key>

CLOUDINARY_API_SECRET=<your-api-secret>

STRIPE_SECRET_KEY=<your-stripe-key>

STRIPE_WEBHOOK_SECRET=<your-webhook-secret>

Do not commit this file.

**Build the Images**

docker compose build

**Start the Application**

docker compose up -d

**Check Container Status**

docker compose ps

Expected services:

greencart-mongodb

greencart-backend

greencart-frontend

**View Logs**

Backend:

docker compose logs --tail=50 backend

MongoDB:

docker compose logs --tail=50 mongodb

Frontend:

docker compose logs --tail=50 frontend

**Stop the Application**

docker compose down

The named MongoDB volume is retained.

To intentionally remove the database volume:

docker compose down -v

Do not use -v when you want to preserve database data.

**Testing**

**Backend Health Check**

curl http://localhost:4000/

Expected response:

Api running

**Product API**

curl http://localhost:4000/api/product/all

The endpoint returns the product collection from MongoDB.

**Frontend**

The frontend is available through Nginx:

http://localhost/

When deployed to EC2:

http://<EC2-PUBLIC-IP>/

**End-to-End API Test**

The public application can also access the backend API through Nginx:

http://<EC2-PUBLIC-IP>/api/product/all

The request path is:

Internet

↓

EC2 :80

↓

Nginx

↓

backend:4000

↓

mongodb:27017

**AWS EC2 Deployment**

The application was deployed on an AWS EC2 Ubuntu instance using Docker Compose.

Deployment steps:

git clone https://github.com/sameersyed7/GreenCart.git

cd GreenCart

docker compose build

docker compose up -d

docker compose ps

The EC2 Security Group allows public HTTP traffic on:

TCP 80

The following ports are not publicly exposed:

TCP 4000

TCP 27017

This keeps the backend and database accessible only through the Docker network.

**Deployment Verification**

The deployment was verified using:

docker compose ps

and:

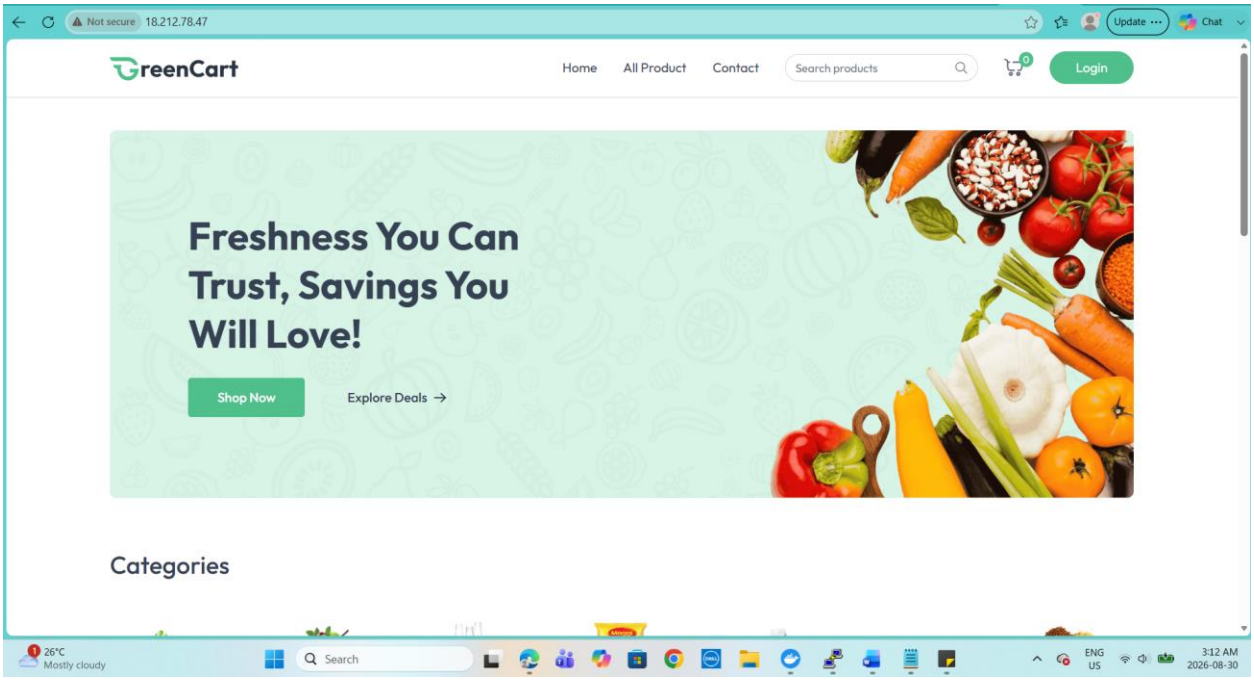
curl -I http://localhost

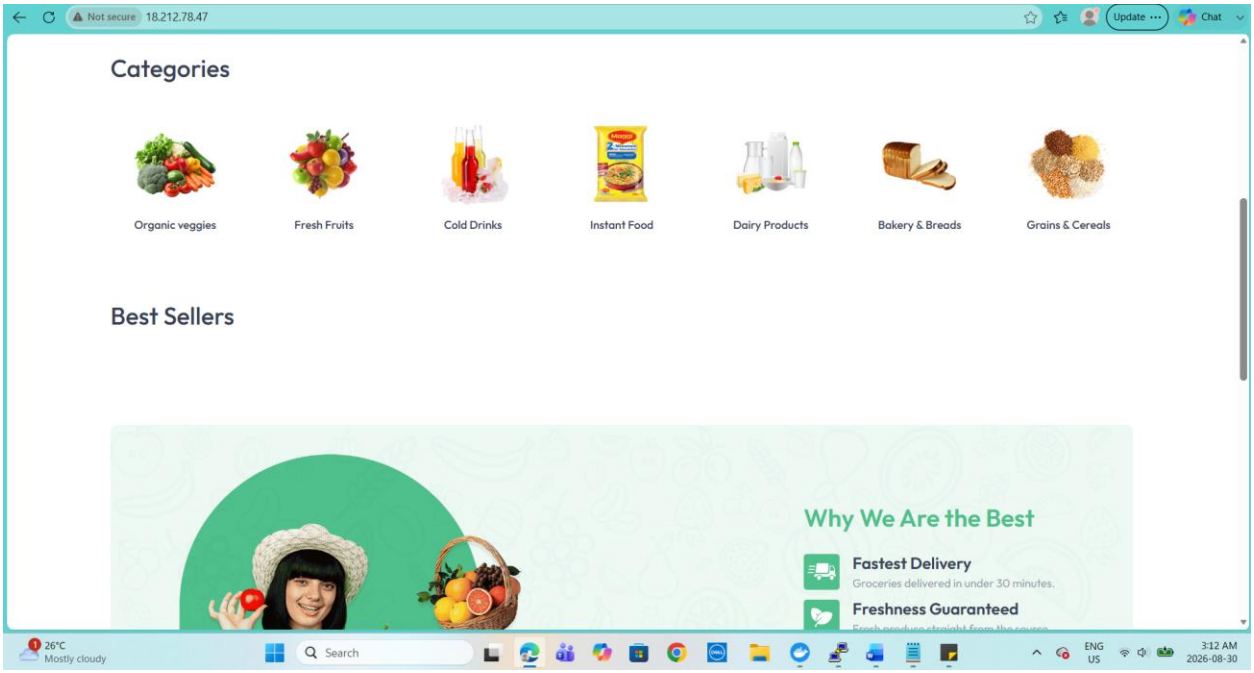
The application was also accessed externally using the EC2 public IP.

Example:

http://<EC2-PUBLIC-IP>

The deployed GreenCart frontend loaded successfully through the public EC2 address.



**Git Practices**

Dockerization was developed through multiple commits rather than a single large commit.

The history includes separate changes for:

- README/project documentation
- Backend Dockerfile
- Docker ignore configuration
- Frontend Docker configuration
- Docker Compose orchestration

Example:

ubuntu@ip-172-31-20-182:~/greenapp/GreenCart$ git log --oneline --decorate -5

182254a (HEAD -> main, origin/main, origin/HEAD) Added docker compose file

0a687cd Add production frontend container configuration

55c5ccc  Added dockerignore file

9a5894e Wrote Dockerfile for backend server & successfully created docker image

ba770ef Update section headers in README.md

9b675e3 Initialize README with project details and instructions

This provides a traceable development history showing the progression of the Dockerization work.

**Repository Structure**

GreenCart/

│

├── client/

│   ├── Dockerfile

│   ├── .dockerignore

```
│   ├── package.json
│   ├── package-lock.json
│   ├── public/
│   └── src/
│
├── server/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── package.json
│   ├── package-lock.json
│   ├── configs/
│   ├── controllers/
│   ├── middlewares/
│   ├── models/
│   └── routes/
│
├── docker-compose.yml
├── .gitignore
└── README.md
```

**Key Dockerization Decisions**

**1. Separate application containers**

Frontend and backend are independently containerized, allowing each component to use an appropriate runtime.

**2. Multi-stage builds**

Build dependencies are separated from production runtime dependencies to reduce unnecessary files and layers.

**3. Alpine base images**

Node.js Alpine images are used for the Node.js application stages to keep the images smaller.

**4. Non-root backend**

The backend production container runs using the non-root node user.

**5. Internal Docker networking**

Services communicate using Docker service names instead of hardcoded IP addresses.

**6. Database isolation**

MongoDB is not exposed to the public Internet.

**7. Persistent storage**

MongoDB uses a named volume so database data survives container recreation.

**8. Secret management**

Credentials and API keys are supplied through environment variables and excluded from Git.

**Assignment Requirements Checklist**

| Requirement | Status |
| --- | --- |
| Public 3-tier application | ✅ |
| Original repository documented | ✅ |
| Frontend container | ✅ |
| Backend container | ✅ |
| Separate Dockerfiles | ✅ |
| Multi-stage builds | ✅ |
| .dockerignore | ✅ |
| Docker Compose | ✅ |

| Requirement | Status |
| --- | --- |
| Custom Docker network | ✅ |
| Service-name communication | ✅ |
| Database port not exposed | ✅ |
| Named database volume | ✅ |
| Persistence tested | ✅ |
| EC2 deployment | ✅ |
| Public application access | ✅ |
| Meaningful Git history | ✅ |
| README documentation | ✅ |
| Image sizes documented | ✅ |
| Architecture diagram | ⏳ Add final diagram |
| Deployment evidence PDF | ⏳ Add final screenshots |

**Evidence**

The final submission includes evidence covering:

1. EC2 instance creation
2. Security Group configuration
3. Docker installation/setup
4. Docker Compose deployment
5. Running containers
6. Public application access
7. MongoDB persistence test

---

**Author**

**Sameer Syed**

GreenCart Dockerization & AWS EC2 Deployment